Fuzail Ayaz Khan
ENG24CY0102
3-B

Linux Programming Assignment 3

1. **A system has a file /etc/passwd. How would you use grep + tee to extract usernames and save them to a file while also displaying them on screen? (CO4)**

   The /etc/passwd file contains user account information in a structured format where each line represents a user account, and the username appears as the first field before the colon delimiter. To extract usernames using grep combined with tee, you need to use grep with a regular expression that matches the beginning of each line up to the first colon, then pipe the results through tee to simultaneously display and save the output. The command would be: grep -o '[:]' */etc/passwd | tee usernames.txt. This approach uses grep with the -o flag to output only the matching portions rather than entire lines, and the regular expression '[:]'* anchors the search to the beginning of each line and matches any sequence of characters that are not colons.

   An alternative approach would be to use cut command in combination with grep and tee for more precise field extraction: cut -d: -f1 /etc/passwd | tee usernames.txt. This method is actually more reliable because cut is specifically designed for field extraction from delimited files. The -d: option specifies the colon as the delimiter, -f1 selects the first field which contains the username, and tee creates a copy of the output stream that goes both to the terminal display and to the specified file usernames.txt. This command effectively demonstrates the power of combining multiple Linux utilities through pipes to achieve complex data processing tasks while maintaining real-time visibility of the results.

2. **A binary isn't found in $PATH. How would you use commands (which, find, locate) to troubleshoot and fix the issue? (CO3)**

   When a binary isn't found in the PATH, systematic troubleshooting involves using multiple search commands to locate the executable and then taking appropriate action to make it accessible. Start by using the which command to confirm that the binary is indeed not in your current PATH by running which binary_name, which will return nothing if the binary isn't found in the directories listed in your PATH environment variable. Next, use the find command to search the entire filesystem or specific directories where the binary might be located with find / -name binary_name -type f 2>/dev/null, which searches from the root directory for files with the exact name while suppressing permission denied errors.

The locate command provides another search method by querying a pre-built database of file locations using locate binary_name, though this requires the database to be updated regularly with updatedb command and might not show recently created files. Once you've located the binary using these commands, you have several options to fix the PATH issue. If the binary is in a standard location like /usr/local/bin or /opt, you can add that directory to your PATH environment variable by editing your shell configuration file such as ~/.bashrc and adding export PATH=$PATH:/path/to/directory, then sourcing the file with source ~/.bashrc to apply the changes immediately.

Alternatively, if the binary is in a non-standard location, you can create a symbolic link to place it in a directory that's already in your PATH using ln -s /full/path/to/binary /usr/local/bin/binary_name, assuming you have the necessary permissions. For temporary use, you can execute the binary using its full path without modifying the PATH. The combination of these search commands provides comprehensive coverage for locating missing binaries, and the various resolution methods offer flexibility depending on whether you need a permanent system-wide solution or a temporary fix for the current session.

3. Write a command pipeline that finds all .log files modified in the last 24 hours in /var/log and saves results into log_report.txt. (CO4)

To create a comprehensive command pipeline that finds all .log files modified within the last 24 hours in the /var/log directory and saves the results to a report file, you need to combine the find command with appropriate time-based filters and output redirection. The complete command would be: find /var/log -name "*.log" -type f -mtime -1 -ls | tee log_report.txt. This pipeline uses find to search the /var/log directory with -name "*.log" to match files ending with the .log extension, -type f to ensure only regular files are included, and -mtime -1 to find files modified within the last 24 hours where -1 means less than 1 day ago.

The -ls option provides detailed output similar to ls -l command, showing file permissions, ownership, size, and modification timestamps, which is valuable information for log analysis. The tee command then splits the output stream, displaying the results on the terminal while simultaneously writing them to log_report.txt. For more detailed information, you could enhance the pipeline with additional formatting: find /var/log -name "*.log" -type f -mtime -1 -exec ls -lh {} ; | tee log_report.txt, which uses

-exec to run ls -lh on each found file, providing human-readable file sizes and detailed timestamps.

If you want to include additional context such as the total count of files found, you could extend the pipeline further: find /var/log -name "*.log" -type f -mtime -1 -ls | tee log_report.txt && echo "Total files found: $(find /var/log -name "*.log" -type f -mtime -1 | wc -l)" | tee -a log_report.txt. This approach not only creates a comprehensive log of recently modified log files but also provides immediate visibility of the results while building a permanent record that can be used for system administration, troubleshooting, or compliance purposes.

4. What is the difference between shutdown -r now and reboot? (CO1)

The commands shutdown -r now and reboot serve the same fundamental purpose of restarting a Linux system, but they differ in their approach, timing mechanisms, and historical implementation. The shutdown -r now command is part of the comprehensive shutdown utility that provides extensive control over the shutdown process, including the ability to schedule shutdowns for specific times, broadcast warning messages to logged-in users, and gracefully terminate all running processes. The -r flag specifically instructs the system to reboot after shutdown, while now indicates that the action should be performed immediately without delay.

In contrast, the reboot command is designed as a more direct and immediate restart mechanism that typically bypasses some of the scheduling and notification features of shutdown. Historically, reboot was a more low-level command that could potentially force a restart with less graceful handling of running processes, though modern implementations have largely standardized the behavior between these commands. On systemd-based distributions, which include most contemporary Linux systems, both commands ultimately map to the same underlying systemctl operations, making their practical differences minimal in terms of the actual restart process.

The key distinctions lie in their syntax flexibility and additional features. The shutdown command allows for more sophisticated scheduling options such as shutdown -r +10 "System will restart in 10 minutes" which provides advance notice to users, while reboot is generally intended for immediate action. Additionally, shutdown provides options like -c to cancel a pending shutdown, -k to send warning messages without actually shutting down, and various timing specifications. For system administrators, shutdown -r now is

often preferred in scripts and automated scenarios because of its consistent behavior and explicit syntax, while reboot might be used for quick manual restarts. Both commands require appropriate privileges and will gracefully terminate services and unmount filesystems before performing the restart operation.

5. **How can you use the tee command to debug a script that generates both standard output and error messages? (CO4)**

The tee command can be effectively used for script debugging by capturing and displaying both standard output and standard error streams, allowing you to monitor script execution in real-time while preserving all output for later analysis. To capture both streams, you need to redirect stderr to stdout and then pipe the combined output through tee using the command: ./myscript.sh 2>&1 | tee debug_output.log. This approach merges stderr into stdout using 2>&1 redirection, then tee displays the combined output on the terminal while simultaneously writing it to the debug log file.

For more sophisticated debugging where you want to separate stdout and stderr into different files while still displaying both, you can use process substitution with multiple tee commands: ./myscript.sh > >(tee stdout.log) 2> >(tee stderr.log >&2). This advanced technique creates separate process substitutions for each stream, with the first tee handling standard output and writing it to stdout.log, while the second tee processes standard error, writes it to stderr.log, and uses >&2 to ensure error messages still appear on the terminal's error stream.

Another useful debugging approach involves using tee with timestamps to track when different outputs occur during script execution: ./myscript.sh 2>&1 | while IFS= read -r line; do echo "$(date '+%Y-%m-%d %H:%M:%S'): $line"; done | tee timestamped_debug.log. This pipeline adds timestamps to each line of output, making it easier to correlate events and identify timing-related issues in your script. For interactive debugging sessions, you might also use tee with the -a flag to append to existing log files across multiple test runs: ./myscript.sh 2>&1 | tee -a cumulative_debug.log, allowing you to build a comprehensive debugging history while making iterative improvements to your script. These techniques provide comprehensive visibility into script behavior and create permanent records that facilitate thorough debugging and troubleshooting processes.

6. **Explain any three real-world applications of Linux in industries. (CO2)**

Linux has become the backbone of modern server infrastructure across virtually all industries, powering everything from small web servers to massive data centers that serve billions of users worldwide. Major technology companies like Google, Facebook, Amazon, and Netflix rely entirely on Linux-based server farms to deliver their services, with Google alone operating millions of Linux servers to handle search queries, Gmail, YouTube, and cloud services. The reliability, scalability, and cost-effectiveness of Linux make it ideal for enterprise server environments where uptime requirements are critical and licensing costs for proprietary alternatives would be prohibitively expensive. Linux distributions like Red Hat Enterprise Linux, Ubuntu Server, and CentOS provide enterprise-grade stability with long-term support, security patches, and professional technical support that meet the demanding requirements of business-critical applications.

The embedded systems industry heavily relies on Linux for everything from smart appliances and automotive systems to industrial control equipment and Internet of Things devices. Automotive manufacturers increasingly use embedded Linux systems for infotainment systems, navigation, advanced driver assistance systems, and even autonomous driving platforms. Companies like Tesla, BMW, and Ford integrate Linux-based systems throughout their vehicles to manage complex functionality while maintaining real-time performance requirements. Industrial automation and manufacturing equipment extensively use embedded Linux for programmable logic controllers, robotic systems, and process control applications where reliability and real-time capabilities are essential. The flexibility of Linux allows embedded system developers to create highly customized, lightweight operating system configurations that meet specific hardware constraints and functional requirements while avoiding the licensing costs and restrictions associated with proprietary embedded operating systems.

The financial services industry has embraced Linux for high-frequency trading systems, risk management platforms, and core banking applications where performance, security, and reliability are paramount. Major financial institutions like Goldman Sachs, JP Morgan Chase, and Deutsche Bank run their trading platforms on Linux systems that can process thousands of transactions per second with microsecond-level latency requirements. Linux provides the low-latency performance needed for algorithmic trading while offering the security features necessary to protect sensitive financial data and comply with regulatory requirements. Additionally, the film and entertainment industry extensively uses Linux for digital content creation, visual effects rendering, and

post-production workflows. Major studios like Pixar, DreamWorks, and Industrial Light & Magic rely on Linux-based render farms containing thousands of servers to create computer-generated imagery for blockbuster movies, with Linux providing the computational power and stability needed for complex 3D rendering and animation tasks that can take months to complete.

7. **Differentiate application, system and utility software in the context of Linux environment. (CO2)**

In the Linux environment, system software forms the foundational layer that directly interfaces with hardware components and provides essential services for all other software to function. The Linux kernel itself is the core piece of system software, managing hardware resources like CPU scheduling, memory allocation, device drivers, and file system operations. System software in Linux includes the kernel, device drivers that enable communication with specific hardware components, system libraries like glibc that provide essential programming interfaces, and fundamental system services such as systemd for service management, init systems for boot processes, and core networking components. This category of software operates at the lowest level, typically requiring privileged access and running in kernel space or as system processes that start during boot and continue running throughout the system's operation.

Application software in Linux encompasses user-facing programs designed to accomplish specific tasks or solve particular problems for end users. These applications run in user space and depend on system software for accessing hardware resources and system services. Examples include web browsers like Firefox and Chromium, office productivity suites like LibreOffice, media players such as VLC, development environments like Visual Studio Code, graphics editing software like GIMP, and gaming applications. Application software in Linux is typically distributed through package managers like APT for Debian-based systems or YUM for Red Hat-based distributions, and increasingly through universal packaging formats like Snap, Flatpak, or AppImage. These applications interact with users through graphical interfaces or command-line interfaces and are designed to be installed, updated, and removed without affecting the core system functionality.

Utility software in Linux occupies a middle ground between system and application software, providing tools and utilities that enhance system management, maintenance,

and user productivity. Linux utility software includes command-line tools like grep, awk, sed for text processing, find and locate for file searching, tar and gzip for archiving and compression, and system monitoring tools like top, htop, and iostat for performance analysis. Administrative utilities such as fdisk for disk partitioning, cron for task scheduling, and iptables for firewall management are essential for system administration tasks. Many of these utilities are part of core packages like GNU Coreutils, which provides fundamental file, text, and shell utilities that form the foundation of the Linux command-line environment. Unlike application software which serves end-user needs, utility software primarily serves system administrators, developers, and power users who need to perform system maintenance, troubleshooting, or advanced system configuration tasks. These utilities often work together through pipes and shell scripting to create powerful automated workflows and system management solutions.

8. What are the key differences between open-source and proprietary operating systems? (CO2)

The fundamental difference between open-source and proprietary operating systems lies in source code accessibility and licensing models, which creates cascading effects across development processes, cost structures, and user freedoms. Open-source operating systems like Linux distributions make their source code freely available to anyone, allowing users to examine, modify, and redistribute the software according to specific license terms such as the GPL or BSD licenses. This transparency enables independent security audits, community-driven development, and customization to meet specific organizational needs. In contrast, proprietary operating systems like Windows and macOS keep their source code as closely guarded trade secrets, accessible only to the developing company and select partners under strict non-disclosure agreements, which limits user understanding of system functionality and prevents modifications or customizations.

The development and support models differ significantly between these approaches, with open-source systems relying on distributed community collaboration involving thousands of volunteer and professional developers worldwide contributing fixes, features, and improvements. This community-driven model often results in rapid innovation and bug fixes, but support quality can vary depending on community engagement and may lack formal service level agreements. Proprietary systems typically employ centralized development teams with dedicated resources, formal quality assurance processes, and

professional customer support with guaranteed response times and service commitments. However, this centralized approach can result in slower development cycles and may not address the specific needs of all user segments as effectively as the diverse open-source community.

Cost structures and vendor relationships represent another crucial distinction, with open-source operating systems generally available at no licensing cost, though organizations may still incur expenses for professional support, training, or customization services from companies like Red Hat, SUSE, or Canonical. This model eliminates vendor lock-in and provides freedom to switch support providers or handle maintenance internally. Proprietary operating systems typically require licensing fees that can scale significantly with the number of users or devices, creating ongoing financial commitments and potential vendor dependency. Additionally, security and update models differ substantially, as open-source systems benefit from transparent development processes where vulnerabilities can be identified and patched by anyone with sufficient expertise, often resulting in faster security responses. Proprietary systems rely on internal security teams and coordinated disclosure processes, which may provide more controlled security management but can potentially delay fixes for critical vulnerabilities due to limited development resources and corporate priorities.

9. Write the command to display the system's kernel version. (CO3)

The most common and straightforward command to display the Linux kernel version is uname -r, which outputs only the kernel release information in a concise format showing the version number, release level, and distribution-specific details. For example, a typical output might look like "5.15.0-56-generic" where 5.15 represents the major kernel version, 0 indicates the minor version, 56 shows the patch level, and "generic" represents the distribution-specific configuration. This command is universally available across all Linux distributions and provides the essential kernel version information needed for compatibility checks, troubleshooting, or system documentation purposes.

For more comprehensive kernel and system information, you can use uname -a which displays all available system information including the kernel name, network node hostname, kernel release, kernel version with build date, machine hardware architecture, processor type, hardware platform, and operating system name. This extended output provides complete context about the system environment, which is particularly useful for

system administrators who need detailed information for support tickets, hardware compatibility verification, or system inventory documentation. An alternative command uname -v specifically shows the kernel version with compilation date and compiler information, which can be valuable for tracking specific kernel builds and identifying when the kernel was compiled.

Additional methods for obtaining kernel version information include examining the /proc/version file using cat /proc/version, which provides detailed information about the kernel version, compiler used for building the kernel, and build timestamp. The hostnamectl command on systemd-based distributions also displays kernel information alongside other system details like hostname, architecture, and operating system identification. For distributions using package management systems, you can also query installed kernel packages using commands like dpkg -l | grep linux-image on Debian-based systems or rpm -qa | grep kernel on Red Hat-based distributions to see which kernel packages are installed and available for the system.

10. What is the difference between head and tail commands in text processing? (CO1)

The head and tail commands serve complementary functions in Linux text processing, with head displaying the beginning portions of files while tail shows the ending portions, making them essential tools for examining file contents without loading entire files into memory. The head command by default outputs the first 10 lines of specified files, allowing users to quickly preview file contents, examine file headers, or extract initial data from large datasets. You can customize the number of lines displayed using the -n option, such as head -n 5 filename.txt to show only the first five lines, or use head -c 100 filename.txt to display the first 100 bytes rather than lines. This functionality makes head particularly useful for examining log files, configuration files, or data files where the most relevant information appears at the beginning.

The tail command performs the opposite function by displaying the last portion of files, defaulting to the final 10 lines but similarly accepting -n and -c options for customization. Beyond basic file viewing, tail offers unique functionality for real-time monitoring through the -f flag, which continuously displays new lines as they are appended to a file, making it invaluable for monitoring active log files, debugging running applications, or tracking system activities in real-time. The command tail -f /var/log/syslog allows system

administrators to watch system events as they occur, while tail -n 50 -f application.log shows the last 50 lines and continues monitoring for new entries.

These commands become particularly powerful when combined with other Linux utilities through pipes and command chains. For instance, you can extract specific sections from files using combinations like head -n 20 filename.txt | tail -n 5 to display lines 16 through 20, or use them with other text processing tools like grep, sort, or awk for more complex data extraction and analysis. Both commands support multiple file inputs and provide helpful filename headers when processing multiple files simultaneously, though this can be suppressed using the -q option for cleaner output in scripts and automated processing scenarios. The complementary nature of these commands makes them fundamental tools in the Linux text processing toolkit, enabling efficient examination of file contents regardless of file size.